

Jetson Containers

Quickstart Guide

51 GPU-accelerated containers · API reference · n8n integration patterns
NVIDIA Jetson AGX Orin 64GB · JetPack 6.2.2 · L4T 36.5.0 · CUDA 12.6

51

Containers

L4T

36.5.0

JetPack

6.2.2

CUDA

12.6

About This Guide

This guide covers all 51 dustynv/jetson-containers packages compatible with L4T 36.4.0+ (JetPack 6.x), tested against your NVIDIA Jetson AGX Orin 64GB running JetPack 6.2.2 / L4T 36.5.0 / CUDA 12.6. Each entry includes: a description, the docker run command, REST API endpoints (where available), a curl example, and an n8n HTTP Request node integration pattern for building AI pipelines.

Your Hardware	NVIDIA Jetson AGX Orin Developer Kit 64GB
OS	Ubuntu 22.04.5 LTS aarch64
JetPack	6.2.2 (L4T 36.5.0)
CUDA	12.6
cuDNN	9.3.0
TensorRT	10.3.0.30
n8n	Running via Docker on port 5678
Ollama	Running natively on port 11434
llama.cpp	Installed at /home/sergiok/llama.cpp/ (ports 8080, 8081)

n8n Key Tip: Your n8n runs in Docker on port 5678. When calling Jetson services from n8n HTTP Request nodes, use your Jetson's LAN IP address (e.g. 192.168.x.x) rather than 'localhost' — Docker networking keeps them separate. For OpenAI-compatible containers, use the OpenAI Chat Model node with a custom Base URL to enable AI Agent and Chain nodes with your local Jetson LLMs.

Table of Contents

LLM Inference — OpenAI-compatible LLM serving engines

[ollama](#) · [llama_cpp](#) · [vllm](#) · [sglang](#) · [mlc](#) · [tensorrt_llm](#) · [nano_llm](#)

LLM Tooling — Quantization, fine-tuning, and tooling for LLMs

[awq](#) · [bitsandbytes](#) · [flashinfer](#) · [llama-factory](#) · [jetson-copilot](#)

Speech & Audio — Speech recognition, text-to-speech, and audio processing

[speaches](#) · [faster-whisper](#) · [kokoro-tts](#) · [piper-tts](#) · [wyoming-whisper](#)

Vision & Diffusion — Image generation, vision-language models, object detection

[stable-diffusion-webui](#) · [comfyui](#) · [sana](#) · [vila](#) · [vila-microservice](#) · [nanoowl](#) · [nanodb](#) · [lita](#) · [robopoint](#)

Robotics & Simulation — Robot frameworks, simulation, and embodied AI

[ros](#) · [genesis](#) · [lerobot](#) · [isaac-gr00t](#) · [cosmos](#) · [nerfstudio](#)

CUDA & GPU Computing — GPU computing, PyTorch, and CUDA-accelerated libraries

[cuda](#) · [cuda-python](#) · [cupy](#) · [pycuda](#) · [pytorch](#) · [l4t-pytorch](#) · [l4t-ml](#) · [jax](#) · [warp](#) · [vulkan](#)

ML Libraries — ML inference, computer vision libraries, and development tools

[onnxruntime](#) · [opencv](#) · [torchvision](#) · [torchaudio](#) · [torch2trt](#) · [torchao](#) · [jupyterlab](#) · [aim](#)

Smart Home — Home automation and IoT integration

[homeassistant-core](#)

API Quick Reference

All containers with REST or API access, their port, and n8n compatibility:

Container	Port	API Style	OpenAI Compat	n8n Node
ollama	11434	OpenAI-compatible REST	Yes	OpenAI Chat Model
llama_cpp	8080	OpenAI-compatible REST	Yes	OpenAI Chat Model
vllm	8000	OpenAI-compatible REST	Yes	OpenAI Chat Model
sglang	30000	OpenAI-compatible REST	Yes	OpenAI Chat Model
mlc	8080	OpenAI-compatible REST	Yes	OpenAI Chat Model
tensorrt_llm	8000	REST	No	HTTP Request
nano_llm	9000	OpenAI-compatible REST	Yes	OpenAI Chat Model
speeches	8000	OpenAI-compatible REST	Yes	OpenAI Chat Model
faster-whisper	8000	OpenAI-compatible REST	Yes	OpenAI Chat Model
kokoro-tts	8880	OpenAI-compatible REST	Yes	OpenAI Chat Model
stable-diffusion-webui	7860	REST	No	HTTP Request
comfyui	8188	REST	No	HTTP Request
vila	8000	OpenAI-compatible REST	Yes	OpenAI Chat Model
vila-microservice	8000	REST	No	HTTP Request
aim	43800	REST	No	HTTP Request
jupyterlab	8888	REST	No	HTTP Request
homeassistant-core	8123	REST	No	HTTP Request
jetson-copilot	8000	REST	No	HTTP Request
l4t-ml	8888	REST	No	HTTP Request

LLM Inference

OpenAI-compatible LLM serving engines

ollama

[REST API](#)[OpenAI Compatible](#)[Port 11434](#)[LLM Inference](#)

Ollama is the easiest way to run large language models locally on your Jetson AGX Orin. Pull and run models like LLaMA 3, Mistral, Qwen, Gemma, Phi, and DeepSeek with a single command. Provides an OpenAI-compatible REST API and model library with automatic GPU offloading.

Note: Most user-friendly LLM runtime. The 64GB AGX Orin can run 70B models comfortably. Multiple CUDA variants.

Links: [Ollama](#) · [jetson-containers](#)

Base URL: <http://localhost:11434/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	OpenAI-compatible chat
POST	/api/generate	Native generate API
GET	/api/tags	List loaded models
POST	/api/pull	Pull a model

Start Command

```
docker run --runtime nvidia -it -d \  
--name ollama \  
--network host \  
-v ~/.ollama:/root/.ollama \  
dustynv/ollama:r36.4.0
```

cURL / API Example

```
# Pull a model first  
curl http://localhost:11434/api/pull \  
-d '{"name": "llama3.2:3b"}'  
  
# Chat completion (OpenAI-compatible)  
curl http://localhost:11434/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "llama3.2:3b",  
  "messages": [{"role": "user", "content": "Hello from n8n!"}]  
'
```

n8n Integration

Use the OpenAI Chat Model node:

- Credential → API Key: any value (e.g. "ollama")
- Base URL: <http://<jetson-ip>:11434/v1>
- Model: llama3.2:3b (or any pulled model)

Or use HTTP Request node:

- Method: POST
- URL: <http://<jetson-ip>:11434/v1/chat/completions>

- Body: {"model":"llama3.2:3b","messages":[...]}

llama_cpp

REST API

OpenAI Compatible

Port 8080

LLM Inference

llama.cpp is a pure C/C++ LLM inference library with CUDA GPU acceleration and GGUF model format support. Run quantized LLMs (Q4, Q5, Q8) with minimal RAM usage and excellent performance on the Jetson AGX Orin's GPU. Best for low-latency, memory-efficient local LLM inference.

Note: Multiple variants including server, Python bindings, and CUDA 12.8. The server variant provides an OpenAI-compatible API.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8080/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	OpenAI chat completion
POST	/v1/completions	Text completion
POST	/completion	Native completion
GET	/health	Health check

Start Command

```
docker run --runtime nvidia -it -d \
--name llama-server \
--network host \
-v ~/models:/models \
dustynv/llama_cpp:r36.4.0 \
llama-server \
--model /models/llama-3.1-8b-q4.gguf \
--host 0.0.0.0 --port 8080 \
--n-gpu-layers 999 --ctx-size 8192
```

cURL / API Example

```
curl http://localhost:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "llama",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What can you do?"}
  ],
  "max_tokens": 256,
  "temperature": 0.7
}'
```

n8n Integration

- OpenAI Chat Model node:
- Base URL: <http://<jetson-ip>:8080/v1>
 - API Key: any (e.g. "none")
 - Model: llama (any string is accepted)

Note: You already have llama.cpp running at port 8080!
Your existing server at /home/sergiok/llama.cpp/ is

already compatible — just point n8n at it.

vllm

REST API

OpenAI Compatible

Port 8000

LLM Inference

vLLM is a high-throughput and memory-efficient LLM inference and serving engine using PagedAttention. Achieves 24x higher throughput than naive HuggingFace inference. Best choice for serving LLMs to multiple concurrent users on your Jetson AGX Orin with OpenAI-compatible API.

Note: Multiple CUDA variants. PagedAttention makes optimal use of the 64GB unified memory. Supports tensor parallelism, LoRA, and speculative decoding.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8000/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	OpenAI chat completion
POST	/v1/completions	Text completion
POST	/v1/embeddings	Text embeddings
GET	/v1/models	List loaded models
GET	/health	Health check

Start Command

```
docker run --runtime nvidia -it -d \
--name vllm \
--network host \
--shm-size=8g \
-v ~/.cache/huggingface:/root/.cache/huggingface \
dustynv/vllm:r36.4.0 \
python3 -m vllm.entrypoints.openai.api_server \
--model meta-llama/Llama-3.2-3B-Instruct \
--host 0.0.0.0 --port 8000 \
--max-model-len 8192
```

cURL / API Example

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "meta-llama/Llama-3.2-3B-Instruct",
  "messages": [{"role": "user", "content": "Summarize this for n8n."}],
  "max_tokens": 512
}'

# Check available models
curl http://localhost:8000/v1/models
```

n8n Integration

OpenAI Chat Model node:

- Base URL: <http://<jetson-ip>:8000/v1>
- API Key: "token-abc" (any string)
- Model: meta-llama/Llama-3.2-3B-Instruct

Supports streaming — enable in HTTP Request node with "Response Format: Stream".

sglang

REST API

OpenAI Compatible

Port 30000

LLM Inference

SGLang (Structured Generation Language) is a fast inference and serving framework for LLMs with advanced scheduling — RadixAttention for KV-cache reuse, continuous batching, and multi-LoRA serving. Achieves very high throughput for serving LLMs with structured outputs on Jetson.

Note: Best for high-throughput multi-user LLM serving scenarios. Provides OpenAI-compatible API.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:30000/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	OpenAI chat completion
POST	/v1/completions	Text completion
GET	/v1/models	List models
GET	/health	Health check

Start Command

```
docker run --runtime nvidia -it -d \
--name sglang \
--network host \
--shm-size=8g \
-v ~/.cache/huggingface:/root/.cache/huggingface \
dustyntv/sglang:r36.4.0 \
python3 -m sglang.launch_server \
--model-path meta-llama/Llama-3.2-3B-Instruct \
--host 0.0.0.0 --port 30000
```

cURL / API Example

```
curl http://localhost:30000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "default",
  "messages": [{"role": "user", "content": "Hello!"}]
}'
```

n8n Integration

OpenAI Chat Model node:

- Base URL: `http://<jetson-ip>:30000/v1`
- API Key: any value
- Model: default (or actual model name)

SGLang excels at structured outputs — use `response_format: {"type": "json_object"}` in n8n HTTP Request for JSON-constrained generation.

mlc

REST API

OpenAI Compatible

Port 8080

LLM Inference

MLC LLM (Machine Learning Compilation) by TVM compiles LLMs to optimized native GPU kernels specific to your hardware. Achieves the fastest raw token generation speeds on Jetson by compiling models directly for the Orin GPU architecture — often faster than llama.cpp or vLLM.

Note: Fastest LLM inference option for Jetson. Models must be compiled or downloaded pre-compiled. Multiple CUDA variants.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8080/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	OpenAI chat completion
GET	/v1/models	List models

Start Command

```
docker run --runtime nvidia -it -d \  
--name mlc \  
--network host \  
--shm-size=8g \  
-v ~/.cache/mlc:/root/.cache/mlc \  
dustyntv/mlc:r36.4.0 \  
python3 -m mlc_llm.serve \  
HF://mlc-ai/Llama-3.2-3B-Instruct-q4f16_1-MLC \  
--host 0.0.0.0 --port 8080
```

cURL / API Example

```
curl http://localhost:8080/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "HF://mlc-ai/Llama-3.2-3B-Instruct-q4f16_1-MLC",  
  "messages": [{"role": "user", "content": "Hi!"}],  
  "max_tokens": 128  
'
```

n8n Integration

OpenAI Chat Model node:

- Base URL: <http://<jetson-ip>:8080/v1>
- Model: the MLC model path (e.g. HF://mlc-ai/...)

MLC achieves the fastest token generation speed —
ideal for time-sensitive n8n automation pipelines.

tensorrt_llm

REST API

Port 8000

LLM Inference

TensorRT-LLM is NVIDIA's high-performance LLM inference library that compiles LLMs to optimized TensorRT engines. Uses in-flight batching, paged KV-cache, and FP8/INT4 quantization to achieve maximum throughput on Jetson. The most hardware-optimized LLM inference option from NVIDIA.

Note: Models must be compiled to TensorRT engines first. Complex setup but highest NVIDIA-certified performance.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8000>

Method	Endpoint	Description
POST	/v2/models/ensemble/generate	Generate (Triton API)
GET	/v2/health/ready	Health check

Start Command

```
docker run --runtime nvidia -it -d \
--name tensorrt-llm \
--network host \
--shm-size=8g \
-v ~/trt-engines:/engines \
dustynv/tensorrt_llm:r36.4.0
```

cURL / API Example

```
# TensorRT-LLM uses Triton Inference Server protocol
curl http://localhost:8000/v2/models/ensemble/generate \
-H "Content-Type: application/json" \
-d '{
"text_input": "Hello from Jetson!",
"max_tokens": 100,
"bad_words": "",
"stop_words": ""
}'
```

n8n Integration

Use HTTP Request node (not OpenAI node):

- Method: POST
- URL: <http://<jetson-ip>:8000/v2/models/ensemble/generate>
- Body Type: JSON
- Body: `{"text_input":"{{$json.input}}","max_tokens":256}`

Extract response with Expression: `{{$json.text_output}}`

nano_llm

REST API

OpenAI Compatible

Port 9000

LLM Inference

NanoLLM is NVIDIA's optimized LLM inference library for Jetson, providing a simple Python API over MLC LLM with multimodal capabilities. Supports vision-language models (VILA, LLaVA, InternVL), voice pipelines, and agent workflows designed for Jetson edge deployment.

Note: The highest-level abstraction for multimodal AI on Jetson. Great for building complete AI pipelines.

Links: [Jetson AI Lab](#) · [jetson-containers](#)

Base URL: <http://localhost:9000/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	OpenAI chat (vision-language)
GET	/v1/models	List models

Start Command

```
docker run --runtime nvidia -it -d \  
--name nano-llm \  
--network host \  
--shm-size=8g \  
-v ~/.cache/huggingface:/root/.cache/huggingface \  
dustynv/nano_llm:r36.4.0 \  
python3 -m nano_llm.serve \  
--model Efficient-Large-Model/VILA1.5-3b \  
--host 0.0.0.0 --port 9000
```

cURL / API Example

```
# Text + Image (vision-language)  
curl http://localhost:9000/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "VILA1.5-3b",  
  "messages": [{  
    "role": "user",  
    "content": [  
      {"type": "image_url",  
        "image_url": {"url": "http://example.com/img.jpg"}},  
      {"type": "text", "text": "What is in this image?"}  
    ]  
  }]  
}'
```

n8n Integration

HTTP Request node (multimodal):

- Method: POST
- URL: `http://<jetson-ip>:9000/v1/chat/completions`
- Body: JSON with content array (text + image_url)

Use NanoLLM for vision tasks in n8n pipelines —
analyze screenshots, camera frames, or document images.

LLM Tooling

Quantization, fine-tuning, and tooling for LLMs

awq

LLM Tooling

AWQ (Activation-aware Weight Quantization) is an efficient LLM quantization method that achieves 4-bit weight quantization with minimal accuracy loss. Enables running larger language models within the 64GB unified memory of the Jetson AGX Orin by reducing VRAM footprint by ~4x.

Note: Requires PyTorch. Works alongside transformers and vLLM for quantized inference.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python library](#)

Start Command

```
jetson-containers run dustynv/awq:r36.4.0
```

cURL / API Example

```
# Python library – no REST API
# Use as preprocessing step to quantize models
# before loading into vLLM or other servers
```

n8n Integration

No direct REST API. Used in Python scripts to pre-quantize LLMs.
Resulting quantized models load into vLLM for API serving.

bitsandbytes

LLM Tooling

bitsandbytes enables accessible large-scale model inference and fine-tuning using k-bit quantization (4-bit and 8-bit) via CUDA kernels. Essential for loading large LLMs like LLaMA, Mistral, and Falcon in NF4/INT8 precision on the Jetson AGX Orin.

Note: Provides QLoRA support for on-device fine-tuning. Works seamlessly with HuggingFace transformers and PEFT.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python library](#)

Start Command

```
jetson-containers run dustynv/bitsandbytes:0.45.4-r36.4.0-cu128-24.04
```

cURL / API Example

```
# Python library – no REST API
# Used with HuggingFace transformers
# for 4-bit/8-bit model loading
```

n8n Integration

No direct REST API. Used with transformers for 4-bit/8-bit model loading.
Load quantized models into a REST server (e.g. vLLM) for n8n access.

flashinfer

LLM Tooling

FlashInfer is a library of GPU kernels for LLM serving — providing highly optimized attention operations (FlashAttention, PagedKV, RadixAttention) and KV-cache management. Used by vLLM and SGLang as their inference kernel backend on Jetson.

Note: Low-level kernel library; typically used as a dependency of vLLM/SGLang rather than standalone.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/flashinfer:0.2.3-r36.4.0-cu128-24.04
```

cURL / API Example

```
# Low-level attention kernel library
# Dependency for vLLM and SGLang
# Not used directly – no REST API
```

n8n Integration

No REST API. Backend kernel library used by vLLM and SGLang automatically.

No direct n8n integration needed.

llama-factory

Port 7860

LLM Tooling

LLaMA-Factory is a unified framework for fine-tuning 100+ LLMs (LLaMA, Mistral, Qwen, Gemma, etc.) using LoRA, QLoRA, full fine-tuning, and RLHF. Run LLM fine-tuning experiments directly on your 64GB Jetson AGX Orin with a web-based training UI.

Note: 64GB unified memory makes the AGX Orin capable of fine-tuning 7B–13B models with QLoRA.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:7860> (Gradio UI)

Method	Endpoint	Description
GET	/	LLaMA-Factory training web UI

Start Command

```
docker run --runtime nvidia -it -d \
--name llama-factory \
--network host --shm-size=16g \
-v ~/.cache/huggingface:/root/.cache/huggingface \
-v ~/datasets:/datasets \
dustynv/llama-factory:r36.4.0 \
llamafactory-cli webui
```

cURL / API Example

```
# Web UI for fine-tuning – access at http://localhost:7860
# No REST API – interactive training interface
```

```
# CLI fine-tuning:
docker exec llama-factory llamafactory-cli train \
--stage sft --model_name_or_path meta-llama/Llama-3.1-8B \
--dataset my_dataset --output_dir /output/fine-tuned
```

n8n Integration

No REST API. Use Gradio web UI at port 7860 for interactive fine-tuning.

After fine-tuning, serve the resulting model via vLLM or llama.cpp for n8n integration.

jetson-copilot

REST API

Port 8000

LLM Tooling

Jetson Copilot is an on-device AI assistant that runs entirely locally on your Jetson AGX Orin — no cloud required. Provides a chat interface backed by local LLMs via Ollama, with optional RAG (Retrieval-Augmented Generation) over your own documents.

Note: Includes a ready-to-use web UI. Models are downloaded on first use via Ollama.

Links: [Jetson AI Lab](#) · [jetson-containers](#)

Base URL: <http://localhost:8000>

Method	Endpoint	Description
GET	/	Jetson Copilot chat web UI

Start Command

```
jetson-containers run \
-p 8000:8000 \
-v ~/jetson-copilot:/data \
-v ~/.ollama:/root/.ollama \
dustynv/jetson-copilot:r36.4.0
```

cURL / API Example

```
# Chat UI at http://localhost:8000
# Backed by Ollama — use Ollama API directly:
curl http://localhost:11434/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{"model": "llama3.2:3b", "messages": [{"role": "user", "content": "Hello"}]}'
```

n8n Integration

Use the underlying Ollama API (port 11434) for n8n integration.

Jetson Copilot is the UI layer — the Ollama API behind it is fully n8n-compatible.

See Ollama entry for n8n setup.

Speech & Audio

Speech recognition, text-to-speech, and audio processing

speaches

[REST API](#)[OpenAI Compatible](#)[Port 8000](#)[Speech & Audio](#)

Speaches is a self-hosted OpenAI-compatible Speech API server providing text-to-speech and speech-to-text endpoints. Drop-in replacement for the OpenAI audio API — connect any OpenAI-compatible client to your Jetson AGX Orin for on-device speech processing.

Note: Provides `/v1/audio/speech` and `/v1/audio/transcriptions` endpoints compatible with OpenAI client libraries.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8000>

Method	Endpoint	Description
POST	<code>/v1/audio/transcriptions</code>	Speech-to-text (Whisper)
POST	<code>/v1/audio/speech</code>	Text-to-speech (Kokoro/Piper)
GET	<code>/v1/models</code>	List available models
WS	<code>/v1/realtime</code>	WebSocket real-time API

Start Command

```
docker run --runtime nvidia -it -d \  
--name speaches \  
--network host \  
-v ~/.cache/huggingface:/root/.cache/huggingface \  
dustynv/speaches:r36.4.0
```

cURL / API Example

```
# Transcribe audio file (STT)  
curl http://localhost:8000/v1/audio/transcriptions \  
-F file=@recording.wav \  
-F model=Systran/faster-whisper-large-v3  
  
# Generate speech (TTS)  
curl http://localhost:8000/v1/audio/speech \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "kokoro",  
  "input": "Hello from your Jetson AI pipeline!",  
  "voice": "af_bella"  
}' \  
--output speech.mp3
```

n8n Integration

For STT (transcription) — HTTP Request node:

- Method: POST
- URL: `http://<jetson-ip>:8000/v1/audio/transcriptions`
- Body Type: Form-Data
- Field "file": Binary audio data (from Read Binary node)
- Field "model": Systran/faster-whisper-large-v3

- Extract: `{{ $json.text }}`

For TTS — HTTP Request node:

- Method: POST
- URL: `http://<jetson-ip>:8000/v1/audio/speech`
- Body: `{"model":"kokoro","input":"{{ $json.text }}","voice":"af_bella"}`
- Response Format: File

faster-whisper

REST API

OpenAI Compatible

Port 8000

Speech & Audio

Faster Whisper is a reimplementation of OpenAI's Whisper ASR model using CTranslate2, running up to 4x faster with lower GPU memory usage. Transcribe audio and speech in real-time on the Jetson AGX Orin with INT8 quantization support.

Note: Excellent for on-device voice transcription, live captioning, and voice command pipelines.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: `http://localhost:8000`

Method	Endpoint	Description
POST	<code>/v1/audio/transcriptions</code>	OpenAI-compatible transcription
POST	<code>/v1/audio/translations</code>	Translate audio to English

Start Command

```
docker run --runtime nvidia -it -d \
--name faster-whisper \
--network host \
-v ~/.cache/huggingface:/root/.cache/huggingface \
dustynv/faster-whisper:r36.4.0 \
python3 -m faster_whisper.server \
--host 0.0.0.0 --port 8000 \
--model large-v3 --device cuda
```

cURL / API Example

```
# Transcribe an audio file
curl http://localhost:8000/v1/audio/transcriptions \
-F file=@audio.wav \
-F model=whisper-1 \
-F language=en \
-F response_format=json

# Response:
# {"text": "Transcribed text appears here..."}
```

n8n Integration

- HTTP Request node:
- Method: POST
 - URL: `http://<jetson-ip>:8000/v1/audio/transcriptions`
 - Body Type: Form-Data
 - Add field "file" → Binary (from HTTP Request or Read File node)

- Add field "model" → "whisper-1"
- Parse response: {{\$.json.text}}

Use in voice automation: webhook → record audio →
send to Whisper → feed text to LLM → respond.

kokoro-tts

REST API

OpenAI Compatible

Port 8880

Speech & Audio

Kokoro is a fast, lightweight text-to-speech (TTS) model with high-quality voice synthesis. Generate natural-sounding speech locally on your Jetson AGX Orin — no cloud API needed. Supports multiple voices and languages with low latency suitable for real-time applications.

Note: Multiple variant tags available (CPU/GPU). Ideal for voice assistants and edge TTS applications.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8880>

Method	Endpoint	Description
POST	/v1/audio/speech	OpenAI-compatible TTS
GET	/v1/models	List available models
GET	/v1/voices	List available voices

Start Command

```
docker run --runtime nvidia -it -d \
--name kokoro-tts \
--network host \
dustynv/kokoro-tts:r36.4.0
```

cURL / API Example

```
# Generate speech
curl http://localhost:8880/v1/audio/speech \
-H "Content-Type: application/json" \
-d '{
  "model": "kokoro",
  "input": "Hello! I am your local AI assistant.",
  "voice": "af_bella",
  "response_format": "mp3",
  "speed": 1.0
}' \
--output output.mp3

# Available voices: af_bella, af_sarah, bm_george,
# af_nicole, am_adam, bf_emma, bm_lewis
```

n8n Integration

- HTTP Request node:
- Method: POST
 - URL: <http://<jetson-ip>:8880/v1/audio/speech>
 - Body: {
"model": "kokoro",

```
"input": "{{${json.text}}",
"voice": "af_bella"
}
```

- Response Format: File (binary audio)

Chain: LLM generates text → Kokoro speaks it
→ Send audio file via Telegram/email/webhook.

piper-tts

Port 10200

Speech & Audio

Piper is a fast, local neural text-to-speech system optimized for embedded devices. Generate natural speech from text in real-time on your Jetson AGX Orin with 20+ voice models across multiple languages. Ideal for voice assistants, robotics, and accessibility applications.

Note: Extremely lightweight and fast. Two variants (standard + GPU). Integrates with Wyoming Home Assistant protocol.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: N/A – Wyoming protocol

Method	Endpoint	Description
TCP	port 10200	Wyoming protocol (Home Assistant native)

Start Command

```
docker run --runtime nvidia -it -d \
--name piper-tts \
--network host \
-v ~/piper-models:/models \
dustynv/piper-tts:r36.4.0 \
python3 -m wyoming_piper \
--piper /usr/bin/piper \
--data-dir /models \
--download-dir /models \
--voice en_US-lessac-medium
```

cURL / API Example

```
# Piper TTS uses Wyoming protocol, not REST.
# For n8n, run a REST wrapper instead:

# Option 1: Use speeches (includes Piper backend)
docker run --runtime nvidia -it -d \
--name speeches --network host \
dustynv/speeches:r36.4.0
# Then use /v1/audio/speech with model=piper

# Option 2: Python wrapper script
python3 -c "
import subprocess, sys
text = sys.argv[1]
subprocess.run(['piper', '--model', 'en_US-lessac-medium.onnx',
'--output_file', 'out.wav'], input=text.encode())
" "Hello from Jetson!"
```

n8n Integration

Piper uses Wyoming protocol — not directly REST-callable.

Recommended approach for n8n:

1. Use speeches container (port 8000) instead — it wraps Piper with OpenAI-compatible /v1/audio/speech
2. OR run Piper via shell command in n8n Execute node

Home Assistant integration: wyoming-piper connects directly as a voice assistant TTS provider.

wyoming-whisper

Port 10300

Speech & Audio

Wyoming Whisper provides the Wyoming protocol server wrapping OpenAI Whisper for speech recognition. Integrates directly with Home Assistant's voice pipeline and other Wyoming-compatible clients for local, private speech recognition on your Jetson AGX Orin.

Note: Designed to work with Home Assistant's voice assistant pipeline. Listens on port 10300 by default.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: N/A — Wyoming protocol

Method	Endpoint	Description
TCP	port 10300	Wyoming protocol (Home Assistant native)

Start Command

```
docker run --runtime nvidia -it -d \
--name wyoming-whisper \
--network host \
-v ~/whisper-models:/data \
dustynv/wyoming-whisper:r36.4.0 \
python3 -m wyoming_faster_whisper \
--model large-v3 --language en \
--uri tcp://0.0.0.0:10300 \
--data-dir /data --download-dir /data
```

cURL / API Example

```
# Wyoming Whisper uses TCP protocol, not REST.
# For n8n REST access, use faster-whisper or speeches:

# speeches (port 8000) — full OpenAI audio API
curl http://localhost:8000/v1/audio/transcriptions \
-F file=@audio.wav \
-F model=Systran/faster-whisper-large-v3

# Home Assistant → add as voice assistant STT:
# Settings > Voice Assistants > Add assistant
# Speech-to-text: Wyoming (wyoming-whisper at port 10300)
```

n8n Integration

Wyoming Whisper is not REST-accessible directly.

For n8n voice pipelines, use faster-whisper or speaches containers instead — both expose /v1/audio/transcriptions as a REST endpoint.

Wyoming Whisper is purpose-built for Home Assistant voice assistants — pair it with piper-tts on port 10200 for a full local voice assistant stack.

Vision & Diffusion

Image generation, vision language models, object detection

stable-diffusion-webui

REST API

Port 7860

Vision & Diffusion

Automatic1111's Stable Diffusion Web UI — the most popular image generation UI with support for SD 1.x, SD 2.x, SDXL, and many community extensions. Run locally on your Jetson AGX Orin with GPU acceleration, ControlNet, LoRA, upscaling, and 100+ extensions.

Note: Full-featured web UI. Models download on first use. Extensions stored in mounted volume persist across restarts.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:7860>

Method	Endpoint	Description
POST	/sdapi/v1/txt2img	Text-to-image generation
POST	/sdapi/v1/img2img	Image-to-image generation
GET	/sdapi/v1/models	List available models
POST	/sdapi/v1/options	Get/set server options
GET	/sdapi/v1/progress	Check generation progress

Start Command

```
docker run --runtime nvidia -it -d \
--name sd-webui \
--network host \
-v ~/sd-models:/data/models/stable-diffusion \
-v ~/sd-output:/data/output \
dustynv/stable-diffusion-webui:r36.4.0 \
python3 launch.py \
--data=/data/models/stable-diffusion \
--listen --port=7860 \
--api --nowebui
```

cURL / API Example

```
# Generate an image (requires --api flag at start)
curl http://localhost:7860/sdapi/v1/txt2img \
-H "Content-Type: application/json" \
-d '{
  "prompt": "a futuristic robot on Mars, cinematic",
  "negative_prompt": "blurry, low quality",
  "steps": 20,
  "width": 512,
  "height": 512,
  "cfg_scale": 7,
  "sampler_name": "DPM++ 2M Karras"
}'

# Response: {"images": ["<base64-encoded PNG>"], ...}
# Decode: echo "<base64>" | base64 -d > output.png
```

n8n Integration

HTTP Request node:

- Method: POST
- URL: `http://<jetson-ip>:7860/sdapi/v1/txt2img`
- Body: {
"prompt": "{{ \$json.prompt }}",
"steps": 20, "width": 512, "height": 512
}
- Parse image: {{ \$json.images[0] }} (base64)

Convert base64 to binary file:

- Move Binary Data node → set base64 from expression
- Write Binary File / Send via Telegram

comfyui

REST API

Port 8188

Vision & Diffusion

ComfyUI is a powerful, modular stable diffusion GUI and backend with a graph/node-based workflow system. Run Stable Diffusion XL, FLUX, and other diffusion models locally on your Jetson AGX Orin with GPU acceleration. Full ControlNet, LoRA, and custom node ecosystem support.

Note: 9 GB image includes PyTorch + diffusers stack. Web UI on port 8188.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:8188>

Method	Endpoint	Description
POST	/prompt	Queue a workflow
GET	/history/{id}	Get generation results
GET	/view	Download output image
GET	/queue	Check queue status
WS	/ws	WebSocket for real-time updates

Start Command

```
docker run --runtime nvidia -it -d \
--name comfyui \
--network host \
-v ~/comfyui-models:/root/ComfyUI/models \
-v ~/comfyui-output:/root/ComfyUI/output \
dustyntv/comfyui:r36.4.0
```

cURL / API Example

```
# Step 1: Queue a workflow
PROMPT_ID=$(curl -s http://localhost:8188/prompt \
-H "Content-Type: application/json" \
-d '{"prompt": <workflow_json>, "client_id": "n8n"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['prompt_id'])")

# Step 2: Poll for completion
curl http://localhost:8188/history/$PROMPT_ID
```

```
# Step 3: Download image
curl "http://localhost:8188/view?filename=output.png&type=output" \
--output generated.png
```

n8n Integration

Use HTTP Request node (3-step workflow):

1. POST /prompt → parse {{\$json.prompt_id}}
2. GET /history/{{\$json.prompt_id}} → wait + parse
filename from outputs[0].images[0].filename
3. GET /view?filename={{\$json.filename}}&type=output
→ binary image

Tip: Use a Loop node to poll /history until status is complete (check for non-empty outputs key).

sana

Port 7860

Vision & Diffusion

SANA is an efficient text-to-image generation model by NVIDIA optimized for edge devices. Generate high-quality 1024x1024 images from text prompts on your Jetson AGX Orin faster than SDXL, using a lightweight linear diffusion transformer architecture.

Note: Significantly faster than SDXL on Jetson. Good balance of quality and speed for edge image generation.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:7860> (Gradio UI only)

Method	Endpoint	Description
GET	/	Gradio web interface

Start Command

```
docker run --runtime nvidia -it -d \
--name sana \
--network host \
--shm-size=8g \
-v ~/.cache/huggingface:/root/.cache/huggingface \
dustynv/sana:r36.4.0
```

cURL / API Example

```
# Sana exposes a Gradio web UI – use the browser at:
# http://<jetson-ip>:7860

# For programmatic use, use the Gradio Python client:
pip install gradio-client
python3 -c "
from gradio_client import Client
client = Client('http://localhost:7860')
result = client.predict(
prompt='a beautiful landscape',
api_name='/generate'
)
print(result)
```



```
"
```

n8n Integration

Sana has no native REST API — use Gradio client.

Option 1: Run a Python script in n8n Execute node using the `gradio_client` library to call Sana.

Option 2: Create a thin Flask wrapper:

```
from flask import Flask, request
from gradio_client import Client
app = Flask(__name__)
client = Client('http://localhost:7860')
@app.route('/generate', methods=['POST'])
def gen():
    r = client.predict(request.json['prompt'])
    return {'image_path': r}
app.run(port=7861)
```

vila

REST API

OpenAI Compatible

Port 8000

Vision & Diffusion

VILA (Visual Language Action) is NVIDIA's efficient vision-language model for Jetson — processes images, video, and text with a lightweight architecture optimized for edge deployment. Supports multi-image reasoning, long-context video understanding, and real-time vision-language tasks on your Orin.

Note: NVIDIA's recommended VLM for Jetson. Runs via MLC LLM backend for maximum performance.

Links: [NVIDIA VILA](#) · [jetson-containers](#)

Base URL: <http://localhost:8000/v1>

Method	Endpoint	Description
POST	/v1/chat/completions	Vision-language chat (images + text)
GET	/v1/models	List available models

Start Command

```
docker run --runtime nvidia -it -d \
--name vila \
--network host \
--shm-size=8g \
-e HUGGING_FACE_HUB_TOKEN=<your_token> \
-v ~/.cache/huggingface:/root/.cache/huggingface \
dustynv/vila:r36.4.0 \
python3 -m vila.serve.gradio_web_server_video \
--controller http://localhost:10000 \
--model-list-mode reload
```

cURL / API Example

```
# Vision + Language query
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "Efficient-Large-Model/VILA1.5-3b",
  "messages": [{
    "role": "user",
    "content": [
      {"type": "image_url",
       "image_url": {"url": "http://example.com/photo.jpg"}},
      {"type": "text",
       "text": "Describe what you see in detail."}
    ]
  }],
  "max_tokens": 512
}'
```

n8n Integration

```
HTTP Request node (vision-language):
• URL: http://<jetson-ip>:8000/v1/chat/completions
• Body: {
  "model": "VILA1.5-3b",
  "messages": [{
    "role": "user",
    "content": [
      {"type": "image_url", "image_url": {"url": "{{ $json.imageUrl }}}"},
      {"type": "text", "text": "{{ $json.question }}" }
    ]
  }],
}
```

Use for: image classification in workflows,
reading forms/documents as images, camera feeds.

vila-microservice

REST API

Port 8000

Vision & Diffusion

VILA Microservice packages VILA as a standalone REST API service — exposing vision-language inference as a microservice endpoint. Useful for integrating visual AI capabilities into larger applications, robotics pipelines, or multi-service architectures on Jetson.

Note: REST API server variant of VILA. Designed for service-oriented architectures.

Links: [jetson-containers](#)

Base URL: <http://localhost:8000>

Method	Endpoint	Description
POST	/predict	Image + text inference
GET	/health	Health check

Start Command

```
docker run --runtime nvidia -it -d \
--name vila-microservice \
--network host \
--shm-size=8g \
-v ~/.cache/huggingface:/root/.cache/huggingface \
dustynv/vila-microservice:r36.4.0
```

cURL / API Example

```
# Send image URL + question
curl http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{
  "image": "http://example.com/image.jpg",
  "prompt": "What objects are in this image?"
}'
```

n8n Integration

HTTP Request node:

- Method: POST
- URL: `http://<jetson-ip>:8000/predict`
- Body: {


```
"image": "{{${json.imageUrl}}",
"prompt": "{{${json.question}}"
```
- Parse: `{{${json.response}}`

Microservice version is simpler to integrate
than full VILA — use this for production pipelines.

nanoowl

Port 7860

Vision & Diffusion

NanoOwl is a real-time open-vocabulary object detection model optimized for Jetson using TensorRT. Detects any object you describe in natural language from a live camera feed — no pre-defined class list needed. Uses OWL-ViT compiled to TensorRT for Orin GPU.

Note: TensorRT-optimized for real-time performance. Works with CSI cameras and USB webcams on Jetson.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [N/A](#) — [Python API](#) only

Start Command

```
docker run --runtime nvidia -it --rm \
--name nanoowl \
--network host \
-v /tmp/.X11-unix:/tmp/.X11-unix \
-e DISPLAY=$DISPLAY \
-v ~/nanoowl-models:/models \
dustynv/nanoowl:r36.4.0
```

cURL / API Example

```
# NanoOWL is a Python library — use via Python API
python3 -c "
```

```
from nanoowl.owl_predictor import OwlPredictor
predictor = OwlPredictor(
    'google/owlvit-base-patch32',
    image_encoder_engine='/models/owl_image_encoder_patch32.engine'
)
from PIL import Image
image = Image.open('photo.jpg')
output = predictor.predict(
    image=image,
    texts=['a cat', 'a dog', 'a person'],
    threshold=0.1
)
print(output)
"
```

n8n Integration

NanoOWL has no REST API — must use Python API.

Create a REST wrapper with Flask:

```
from flask import Flask, request
from nanoowl.owl_predictor import OwlPredictor
from PIL import Image
import io, base64

app = Flask(__name__)
predictor = OwlPredictor('google/owlvit-base-patch32')

@app.route('/detect', methods=['POST'])
def detect():
    img_data = base64.b64decode(request.json['image_b64'])
    img = Image.open(io.BytesIO(img_data))
    out = predictor.predict(img, request.json['classes'])
    return {'boxes': str(out.boxes)}

app.run(host='0.0.0.0', port=7861)
```

nanodb

Port 7860

Vision & Diffusion

NanoDB is a lightweight vector database and multimodal embedding store optimized for Jetson. Index and search image/text datasets using CLIP-based embeddings entirely on-device. Enables semantic image search, multimodal RAG, and similarity search without any cloud dependency.

Note: Built on FAISS with GPU-accelerated CLIP embeddings. Designed to run on embedded Jetson hardware.

Links: [Jetson AI Lab](#) · [jetson-containers](#)

Base URL: <http://localhost:7860> (Gradio UI)

Method	Endpoint	Description
GET	/	Gradio image search interface

Start Command

```
docker run --runtime nvidia -it -d \
--name nanodb \
--network host \
-v ~/nanodb-data:/data \
dustynv/nanodb:r36.4.0 \
python3 -m nanodb \
--scan /data/images \
--port 7860 --host 0.0.0.0
```

cURL / API Example

```
# NanoDB runs a Gradio search interface
# Access at http://<jetson-ip>:7860

# For programmatic use - Python API:
from nanodb import NanoDB
db = NanoDB('/data/nanodb.db')
db.scan('/data/images') # Index images
results = db.search('a cat sitting on a couch', k=5)
for r in results:
    print(r.filename, r.score)
```

n8n Integration

NanoDB has a Gradio UI, not a REST API.

Wrap with a simple Flask REST server:

```
from flask import Flask, request
from nanodb import NanoDB

app = Flask(__name__)
db = NanoDB('/data/nanodb.db')

@app.route('/search', methods=['POST'])
def search():
    q = request.json['query']
    k = request.json.get('k', 5)
    results = db.search(q, k=k)
    return {'results': [{'file': r.filename, 'score': r.score}
        for r in results]}

app.run(host='0.0.0.0', port=7861)
```

lita

Vision & Diffusion

LITA (Language Instructed Temporal-Localization Assistant) is a multimodal video understanding model that can localize events in time from natural language descriptions. Useful for video search, event detection, and temporal reasoning on the Jetson AGX Orin.

Note: Builds on LLaVA architecture with temporal awareness. Requires a pre-downloaded model checkpoint.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python Library](#)

Start Command

```
jetson-containers run -v ~/models:/models dustynv/lita:r36.4.0
```

cURL / API Example

```
# Video temporal localization model
# Python API only — no REST server included
```

n8n Integration

No REST API. Wrap video analysis with FastAPI:

```
@app.post('/locate')
```

```
def locate(video, query): return model.predict(video, query)
```

robopoint

Vision & Diffusion

RoboPoint is a vision-language model for robot spatial reasoning — predicting where to place objects or direct robot actions from image + language inputs. Run spatial grounding and manipulation planning on your Jetson AGX Orin for robotic manipulation tasks.

Note: Built on VLM architecture with spatial coordinate output for robot control.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python Library](#)

Start Command

```
jetson-containers run dustynv/robopoint:r36.4.0
```

cURL / API Example

```
# Robot manipulation point prediction model
# Python API for predicting manipulation targets
```

n8n Integration

No REST API. Robot manipulation targeting model. Wrap with FastAPI.

Robotics & Simulation

Robot frameworks, simulation, and embodied AI

ros

Port 11311

Robotics & Simulation

ROS (Robot Operating System) — Humble Hawksbill built for Jetson aarch64. Provides the middleware for robot software development: topics, services, actions, TF2, and the full ROS 2 ecosystem. Three variants: base, core, and desktop (with visualization tools like RViz2).

Note: 3 tag variants for l4t 36.4.0. Desktop variant includes RViz2 for visualization. Use --ipc=host and --network=host for DDS discovery.

Links: [ROS 2 Humble](#) · [jetson-containers](#)

Base URL: [ROS TCP \(port 11311\)](#)

Method	Endpoint	Description
TCP	port 11311	ROS master URI

Start Command

```
docker run --runtime nvidia -it -d \
--name ros \
--network host \
dusty nv/ros:humble-ros-base-r36.4.0
```

cURL / API Example

```
# ROS uses its own middleware, not HTTP REST
# Connect via rosbridge for JSON/WebSocket access:
roslaunch rosbridge_server rosbridge_websocket.launch
# Then connect from n8n via WebSocket at ws://localhost:9090

# Or use ros-rest-api package for HTTP bridge
```

n8n Integration

ROS uses TCP middleware, not REST. For n8n integration:

1. Run rosbridge_server — exposes ws://localhost:9090

2. Use n8n WebSocket node to subscribe/publish

3. Messages are JSON: {"op":"publish","topic":"/cmd","msg":{...}}

Or install ros-rest-api for HTTP-based access:

GET /ros/topics, POST /ros/publish/{topic}

genesis

Robotics & Simulation

Genesis is a physics simulation platform for general-purpose robotics and embodied AI. Provides a unified framework combining rigid-body, soft-body, and fluid simulation with photo-realistic rendering. Run robot simulations and generate synthetic training data directly on your Jetson AGX Orin.

Note: Used for robot policy training data synthesis and sim-to-real transfer research.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/genesis:r36.4.0
```

cURL / API Example

```
import genesis as gs
gs.init(backend=gs.cuda)
scene = gs.Scene()
plane = scene.add_entity(gs.morphs.Plane())
scene.build()
for i in range(100):
    scene.step()
```

n8n Integration

No REST API. Physics simulator. Wrap simulation functions with FastAPI for REST control.

lerobot

Robotics & Simulation

LeRobot (Hugging Face) is a state-of-the-art library for real-world robotics — providing datasets, pre-trained models, and training code for robot learning. Run imitation learning and reinforcement learning policies on your Jetson AGX Orin for robot arm and mobile robot applications.

Note: Multiple environment variants (gym-pusht, gym-aloha, gym-xarm). Integrates with diffusion policy.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/lerobot:r36.4.0
```

cURL / API Example

```
from lerobot.common.policies.diffusion.modeling_diffusion import DiffusionPolicy
policy = DiffusionPolicy.from_pretrained('lerobot/diffusion_pusht')
action = policy.select_action(observation)
```

n8n Integration

No REST API. Robot learning library. Wrap policy inference with FastAPI for REST.

```
@app.post('/act')
def act(obs): return {'action': policy.select_action(obs)}
```

isaac-gr00t

Robotics & Simulation

NVIDIA Isaac GR00T is a generalist robot brain foundation model for humanoid robots. Provides a pre-trained multi-modal transformer that understands language instructions and generates robot actions. Run humanoid robot inference and fine-tuning on your Jetson AGX Orin.

Note: Requires HuggingFace token. Designed for the Isaac robotics ecosystem.

Links: [NVIDIA Isaac GR00T](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run -e HUGGING_FACE_HUB_TOKEN=<token> dustynv/isaac-gr00t:r36.4.0
```

cURL / API Example

```
# Humanoid robot foundation model
# Python API for robot policy inference
# Requires HuggingFace token for model download
```

n8n Integration

No REST API. Humanoid robot brain model. For pipeline integration, wrap action selection with FastAPI.

cosmos

Robotics & Simulation

NVIDIA Cosmos is a world foundation model for physical AI and robotics. It generates physically-plausible video simulations of robot interactions with the real world. Use it for training data synthesis, scenario generation, and robot policy evaluation on your Jetson AGX Orin.

Note: Largest container at 13.16 GB. Requires HuggingFace token for model download.

Links: [NVIDIA Cosmos](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run -e HUGGING_FACE_HUB_TOKEN=<token> dustynv/cosmos:r36.4.0
```

cURL / API Example

```
# World foundation model for robotics
# Generate physically-plausible robot simulation videos
# Requires HuggingFace token
```

n8n Integration

No REST API. Video generation model for robot training data. Very large (13.16 GB). Wrap with FastAPI for REST.

nerfstudio

Port 7007 Robotics & Simulation

Nerfstudio is a modular framework for Neural Radiance Field (NeRF) training and rendering. Reconstruct 3D scenes from photos or video and render novel views — useful for robotics, AR/VR, and 3D mapping tasks runnable on your Jetson AGX Orin.

Note: Supports Nerfacto, Instant-NGP, Splatfacto (Gaussian Splatting), and more. Web viewer included.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:7007> ([Viser 3D viewer](#))

Method	Endpoint	Description
--------	----------	-------------

GET	/	Viser 3D web viewer
-----	---	---------------------

Start Command

```
docker run --runtime nvidia -it --rm \
--name nerfstudio \
--network host \
--shm-size=8g \
-v ~/nerfstudio-data:/data \
-v ~/nerfstudio-output:/output \
dustynv/nerfstudio:r36.4.0 \
ns-train nerfacto \
--data /data/my-scene \
--viewer.websocket-port 7007
```

cURL / API Example

```
# Train NeRF from images:
docker exec nerfstudio ns-train nerfacto \
--data /data/scene \
--output-dir /output

# Render video from trained model:
docker exec nerfstudio ns-render camera-path \
--load-config /output/nerfacto/config.yml \
--camera-path-filename /data/camera_path.json \
--output-path /output/render.mp4

# View 3D reconstruction at http://localhost:7007
```

n8n Integration

NeRFStudio has no REST API.

Use n8n Execute Command node to trigger training:

- Command: `docker exec nerfstudio ns-train nerfacto --data /data/scene`
- Monitor completion via log polling

Or use n8n to orchestrate the pipeline:

- Receive image upload webhook
- Save images to `/data/scene`
- Trigger docker exec training
- Notify when render complete

CUDA & GPU Computing

GPU computing, PyTorch, and CUDA-accelerated libraries

cuda

CUDA & GPU Computing

Base CUDA 12.6 runtime container for the Jetson AGX Orin. Provides the CUDA toolkit headers, libraries, and runtime needed as a foundation layer for all GPU-accelerated workloads. Includes CUDA samples for verifying GPU compute functionality.

Note: Foundational base image (~2.98 GB). Use as FROM in custom Dockerfiles.

Links: [CUDA on Jetson](#) · [jetson-containers](#)

Base URL: [Base container](#)

Start Command

```
jetson-containers run dustynv/cuda:12.8-samples-r36.4.0-cu128-24.04
```

cURL / API Example

```
# Base CUDA image — use as FROM in Dockerfiles
# FROM dustynv/cuda:12.8-samples-r36.4.0-cu128-24.04

# Test GPU:
docker run --runtime nvidia --rm \
dustynv/cuda:12.8-samples-r36.4.0-cu128-24.04 \
/usr/local/cuda/extras/demo_suite/deviceQuery
```

n8n Integration

Base image — no direct n8n integration. Use as Docker base layer.

cuda-python

CUDA & GPU Computing

CUDA Python provides low-level Cython/Python bindings to CUDA APIs, enabling direct GPU programming from Python. Access CUDA driver API, runtime API, and JIT compilation (NVRTC) directly from Python scripts on your Jetson AGX Orin.

Note: Useful for custom CUDA kernels written in Python. Complements CuPy and PyCUDA.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python library](#)

Start Command

```
jetson-containers run dustynv/cuda-python:r36.4.0-cu128-24.04
```

cURL / API Example

```
# Low-level CUDA Python bindings
from cuda import cudart
err, device = cudart.cudaGetDevice()
print(f'Device: {device}')
```

n8n Integration

No REST API. Python library for low-level CUDA programming.

cupy

CUDA & GPU Computing

CuPy is a NumPy/SciPy-compatible array library accelerated with CUDA GPU. Drop-in replacement for NumPy that runs operations on the Jetson's GPU, offering massive speedups for numerical computing, signal processing, and linear algebra.

Note: Ideal for scientific computing workflows that need GPU acceleration without rewriting NumPy code.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python Library](#)

Start Command

```
jetson-containers run dustynv/cupy:r36.4.0-cu128-24.04
```

cURL / API Example

```
import cupy as cp
x = cp.array([1, 2, 3])
print(x * 2) # GPU computation
```

n8n Integration

No REST API. GPU-accelerated NumPy. Use in custom Python processing nodes.

pycuda

CUDA & GPU Computing

PyCUDA gives Python full access to NVIDIA's CUDA parallel computation API with automatic object cleanup, error checking, and meta-programming. Write custom GPU kernels in Python and execute them directly on the Jetson AGX Orin's GPU from Python scripts.

Note: Lower-level than CuPy but more flexible. Used for custom CUDA kernel development and GPU memory management.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python Library](#)

Start Command

```
jetson-containers run dustynv/pycuda:r36.4.0
```

cURL / API Example

```
import pycuda.autoinit
import pycuda.driver as cuda
print('GPU:', cuda.Device(0).name())
```

n8n Integration

No REST API. CUDA programming from Python. Use for custom GPU kernels.

pytorch

CUDA & GPU Computing

PyTorch compiled from source for Jetson aarch64 with full CUDA 12.6, cuDNN 9.x, and TensorRT integration. The foundational deep learning framework — used for training, fine-tuning, and inference of neural networks. Multiple version variants (2.6, 2.7) and CUDA 12.8 builds available.

Note: Your JetPack 6.2.2 is fully compatible with all r36.4.0 PyTorch images. 2.7-r36.4.0 is the latest.

Links: [PyTorch](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/pytorch:r36.4.0
```

cURL / API Example

```
import torch
print(torch.cuda.get_device_name(0))
x = torch.randn(100, 100).cuda()
print(x.sum())
```

n8n Integration

No REST API. Use as a base for building custom model inference servers.

l4t-pytorch

CUDA & GPU Computing

L4T PyTorch is NVIDIA's official pre-built PyTorch container for Jetson, cross-compiled for aarch64 with CUDA, cuDNN, and TensorRT integration. Includes torchvision and torchaudio, optimized specifically for Jetson hardware.

Note: Stable, officially-tested PyTorch build for Jetson. Recommended over building from source.

Links: [NVIDIA PyTorch for Jetson](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/l4t-pytorch:r36.4.0
```

cURL / API Example

```
docker run --runtime nvidia -it --rm --shm-size=8g \
dustynv/l4t-pytorch:r36.4.0 python3 -c "
import torch; print(torch.__version__, torch.cuda.is_available())"
```

n8n Integration

No REST API. NVIDIA's official PyTorch build. Add your inference server code on top.

l4t-ml

REST API

Port 8888

CUDA & GPU Computing

L4T ML is NVIDIA's comprehensive machine learning bundle for Jetson — a batteries-included image with PyTorch, TensorFlow, scikit-learn, JupyterLab, and key ML libraries pre-installed and optimized for L4T. The go-to starting point for ML on Jetson.

Note: All-in-one ML environment. Best choice if you want everything pre-configured without building a stack.

Links: [Jetson AI Lab](#) · [jetson-containers](#)

Base URL: <http://localhost:8888>

Method	Endpoint	Description
GET	/	JupyterLab interface

Start Command

```
docker run --runtime nvidia -it -d \
--name l4t-ml --network host --shm-size=8g \
-v ~/ml-workspace:/workspace \
dustyntv/l4t-ml:r36.4.0 \
jupyter lab --ip=0.0.0.0 --allow-root --no-browser
```

cURL / API Example

```
# JupyterLab on port 8888
# All ML batteries included: PyTorch, TensorFlow, sklearn
# Access at http://<jetson-ip>:8888
```

n8n Integration

Includes JupyterLab — use Jupyter REST API or papermill for notebook execution.
Best used as development environment, not production n8n node.

jax

CUDA & GPU Computing

JAX is Google's high-performance numerical computing library with automatic differentiation and XLA JIT compilation. Supports writing ML research code that runs efficiently on GPU with NumPy-like syntax. Available in both standard and CUDA 12.8 variants.

Note: Useful for ML research, custom optimizer implementations, and scientific computing.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python library](#)

Start Command

```
jetson-containers run dustyntv/jax:r36.4.0-cu128-24.04
```

cURL / API Example

```
import jax
print(jax.devices()) # Should show GPU device
```

n8n Integration

No REST API. JAX for GPU-accelerated research code.

warp

CUDA & GPU Computing

NVIDIA Warp is a Python framework for high-performance simulation and spatial computing using CUDA kernels written in Python. Ideal for physics simulation, differentiable simulation, robotics trajectory optimization, and GPU-accelerated geometry processing on the Jetson AGX Orin.

Note: Most recently updated container (May 2025). Excellent for robotics simulation and GPU-accelerated physics.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/warp:r36.4.0
```

cURL / API Example

```
import warp as wp
wp.init()
@wp.kernel
def hello():
    print('Warp running!')
wp.launch(hello, dim=1, inputs=[])
```

n8n Integration

No REST API. NVIDIA Warp for GPU simulation and physics kernels.

vulkan

CUDA & GPU Computing

Vulkan runtime and SDK for GPU-accelerated graphics and compute on Jetson. Enables cross-platform GPU compute applications, graphics rendering, and frameworks that use Vulkan as their backend (like MoltenVK-based tools). Required by some robotics simulation and 3D rendering pipelines.

Note: Required by certain robotics simulation tools and graphics-accelerated applications on Jetson.

Links: [Vulkan](#) · [jetson-containers](#)

Base URL: [System](#) [Library](#)

Start Command

```
jetson-containers run dustynv/vulkan:r36.4.0
```

cURL / API Example

```
# Vulkan graphics API
vulkaninfo # Check Vulkan support
```

n8n Integration

No REST API. Vulkan for GPU graphics and compute.

ML Libraries

ML inference, computer vision libraries, and development tools

onnxruntime

ML Libraries

ONNX Runtime is a high-performance inference engine for ONNX models with TensorRT and CUDA execution providers. Run PyTorch, TensorFlow, and scikit-learn models converted to ONNX with hardware-accelerated inference on the Jetson AGX Orin's GPU.

Note: Three variants available. TensorRT execution provider gives the best performance for static-shape models.

Links: [ONNX Runtime](#) · [jetson-containers](#)

Base URL: [Python](#) [library](#)

Start Command

```
jetson-containers run dustynv/onnxruntime:r36.4.0
```

cURL / API Example

```
import onnxruntime as ort
sess = ort.InferenceSession('model.onnx', providers=['CUDAExecutionProvider'])
result = sess.run(None, {'input': data})
```

n8n Integration

No REST API. Wrap ONNX model inference with FastAPI for n8n REST access:

```
from fastapi import FastAPI
app = FastAPI()
@app.post('/predict')
def predict(data): return model.run(data)
```

opencv

ML Libraries

OpenCV (Open Computer Vision Library) compiled with CUDA and GStreamer support for Jetson. Enables GPU-accelerated image processing, video I/O via GStreamer CSI camera pipelines, CUDA-optimized filters, and DNN module with GPU inference — all built for aarch64 and L4T.

Note: Your host already has OpenCV 4.8.0. Use this container for projects needing CUDA-enabled OpenCV in isolation.

Links: [OpenCV](#) · [jetson-containers](#)

Base URL: [Python](#) [library](#)

Start Command

```
jetson-containers run dustynv/opencv:r36.4.0
```

cURL / API Example

```
import cv2
cap = cv2.VideoCapture(0) # Camera
ret, frame = cap.read()
cv2.imwrite('frame.jpg', frame)
```


n8n Integration

No REST API. Wrap with Flask/FastAPI:

```
@app.post('/process')
def process(img_b64): frame = decode(img_b64); result = cv2.process(frame); return encode(result)
```

torchvision

ML Libraries

TorchVision provides datasets, model architectures (ResNet, YOLO, DETR, ViT, etc.), and image transforms for computer vision with PyTorch. Built from source for aarch64 with CUDA 12.6 GPU acceleration and video reading support for Jetson.

Note: Two variants: standard and cu128. Required by most PyTorch computer vision projects.

Links: [TorchVision](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/torchvision:r36.4.0
```

cURL / API Example

```
import torchvision
model = torchvision.models.resnet50(pretrained=True).cuda()
model.eval()
```

n8n Integration

No REST API. Use torchvision models via a Flask/FastAPI REST wrapper.

torchaudio

ML Libraries

TorchAudio provides audio and signal processing primitives for PyTorch — audio I/O, spectrograms, filters, and ML models for ASR, TTS, and audio classification. Built for aarch64 with CUDA acceleration for GPU-accelerated audio processing on Jetson.

Note: Required for PyTorch-based audio workflows. Compatible with Whisper and speech recognition pipelines.

Links: [TorchAudio](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/torchaudio:r36.4.0
```

cURL / API Example

```
import torchaudio
waveform, sr = torchaudio.load('audio.wav')
print(waveform.shape, sr)
```

n8n Integration

No REST API. Audio processing library. Wrap with FastAPI for REST.

torch2trt

ML Libraries

torch2trt converts PyTorch models to optimized TensorRT engines with a simple Python API. Automatically applies layer fusion, FP16/INT8 quantization, and Orin-specific optimizations — achieving 2-10x inference speedups for vision models, object detectors, and classifiers.

Note: Essential for deploying any PyTorch model at production speed on Jetson. Two variant tags available.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/torch2trt:r36.4.0
```

cURL / API Example

```
from torch2trt import torch2trt
import torch
model = MyModel().cuda().eval()
x = torch.ones(1, 3, 224, 224).cuda()
model_trt = torch2trt(model, [x])
```

n8n Integration

No REST API. Converts PyTorch models to TensorRT. Resulting TRT model can be served via a FastAPI wrapper.

torchao

ML Libraries

TorchAO (PyTorch Architecture Optimization) provides quantization and sparsity tools integrated directly into PyTorch. Supports INT4, INT8, FP8 weight quantization and activation quantization with torch.compile — enabling smaller model footprints and faster inference on Jetson.

Note: Newer alternative to bitsandbytes with tighter PyTorch integration. Works with torch.compile for extra speedups.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: [Python](#) [Library](#)

Start Command

```
jetson-containers run dustynv/torchao:r36.4.0
```

cURL / API Example

```
from torchao.quantization import quantize_
from torchao.quantization.quant_api import int8_dynamic_activation_int4_weight
quantize_(model, int8_dynamic_activation_int4_weight())
```

n8n Integration

No REST API. Quantization toolkit. Quantize models then serve via vLLM/llama.cpp.

jupyterlab

REST API

Port 8888

ML Libraries

JupyterLab is the next-generation web-based IDE for Jupyter notebooks. Run interactive ML experiments, data analysis, and visualization notebooks directly on your Jetson AGX Orin with full GPU access from the browser-based interface.

Note: Standard and CUDA 12.8 variants available. Essential for interactive ML development on Jetson.

Links: [JupyterLab](#) · [jetson-containers](#)

Base URL: <http://localhost:8888>

Method	Endpoint	Description
GET	/api/kernels	List running kernels
POST	/api/kernels	Start a new kernel
POST	/api/sessions	Create a notebook session

Start Command

```
docker run --runtime nvidia -it -d \
--name jupyterlab \
--network host \
-v ~/notebooks:/notebooks \
dustynv/jupyterlab:r36.4.0 \
jupyter lab --ip=0.0.0.0 --allow-root \
--no-browser --NotebookApp.token=''
```

cURL / API Example

```
# Access at http://<jetson-ip>:8888
# JupyterLab REST API (run notebooks programmatically):
TOKEN=$(docker exec jupyterlab jupyter lab list 2>&1 | grep token | cut -d= -f2)

curl http://localhost:8888/api/kernels \
-H "Authorization: Token $TOKEN"

# Execute code via REST API:
# See: papermill library for notebook execution
```

n8n Integration

Use HTTP Request to trigger notebook execution:

- Jupyter REST API requires a token
- Better: use papermill CLI in Execute Command node:

```
papermill /notebooks/pipeline.ipynb output.ipynb
-p param1 {{$json.value}}
```

Or: use n8n Code node with Python HTTP calls
to JupyterLab kernel API for inline execution.

aim

REST API

Port 43800

ML Libraries

AIM (AI Metadata) is an open-source experiment tracker and ML model metadata store. Track ML training runs, log metrics, hyperparameters, and visualize experiments through a web UI. Compatible with PyTorch, TensorFlow, Keras, and XGBoost.

Note: Lightweight at ~387 MB. Web UI served on port 43800 by default.

Links: [GitHub](#) · [jetson-containers](#)

Base URL: <http://localhost:43800>

Method	Endpoint	Description
GET	/	AIM web dashboard
GET	/api/runs	List training runs
GET	/api/metrics	Query metrics

Start Command

```
docker run --runtime nvidia -it -d \  
--name aim \  
--network host \  
-v ~/aim-data:/data \  
-p 43800:43800 \  
dustynv/aim:3.27.0-r36.4.0 \  
aim server --host 0.0.0.0 --port 43800 --repo /data
```

cURL / API Example

```
# Access AIM dashboard at http://localhost:43800  
# AIM REST API for querying runs:  
curl http://localhost:43800/api/runs \  
-H "Content-Type: application/json"  
  
# Log metrics programmatically from Python:  
from aim import Run  
run = Run(repo='/data')  
run['hparams'] = {'lr': 0.001, 'batch': 32}  
for i in range(100):  
    run.track(loss_val, name='loss', step=i)
```

n8n Integration

HTTP Request node to query training metrics:

- GET <http://<jetson-ip>:43800/api/runs>
- GET <http://<jetson-ip>:43800/api/metrics>

Use n8n to monitor training runs:

- Poll AIM API every 5 min
- Check if loss plateaued
- Send Slack/email notification

Smart Home

Home automation and IoT integration

homeassistant-core

REST API

Port 8123

Smart Home

Home Assistant Core is the open-source home automation platform. Run a full smart home controller locally on your Jetson AGX Orin, integrating with hundreds of IoT devices, sensors, and services — all without cloud dependency.

Note: Persist configuration via volume mount. Web UI on port 8123.

Links: [Home Assistant](#) · [jetson-containers](#)

Base URL: <http://localhost:8123>

Method	Endpoint	Description
GET	/api/states	Get all entity states
POST	/api/services/{domain}/{service}	Call a service
POST	/api/events/{event}	Fire an event
GET	/api/history	Get state history

Start Command

```
docker run --runtime nvidia -it -d \
--name homeassistant \
--network host \
--restart unless-stopped \
-v ~/homeassistant-config:/config \
dustynv/homeassistant-core:r36.4.0

# Get HA token from: Settings > Users > Long-Lived Tokens
```

cURL / API Example

```
# Get all entity states
HA_TOKEN="your_long_lived_access_token"
curl http://localhost:8123/api/states \
-H "Authorization: Bearer $HA_TOKEN" \
-H "Content-Type: application/json"

# Turn on a light
curl http://localhost:8123/api/services/light/turn_on \
-H "Authorization: Bearer $HA_TOKEN" \
-H "Content-Type: application/json" \
-d '{"entity_id": "light.living_room"}
```

n8n Integration

HTTP Request node with HA REST API:

- URL: `http://<jetson-ip>:8123/api/states`
- Auth: Header → Authorization: Bearer <token>

Powerful combo: n8n ↔ Home Assistant:

- HA triggers n8n webhook on sensor change
- n8n calls LLM to analyze conditions

→ n8n calls HA /api/services to control devices

Create Long-Lived Token in HA:

Profile → Long-Lived Access Tokens → Create

Appendix: n8n Integration Patterns

OpenAI-Compatible LLM Nodes

All containers marked OpenAI Compatible can be used with the n8n OpenAI Chat Model node (used in AI Agent and Chain nodes). Go to: Credentials → Add → OpenAI → set Base URL to your Jetson's LAN IP and port. Any API key value works (it is not validated).

```
# n8n OpenAI Chat Model node settings:
# 1. Add Credential: OpenAI API → Custom Base URL
# 2. API Key: "local" (any string)
# 3. Base URL: http://192.168.x.x:<port>/v1

# Container → Base URL mapping:
# ollama → http://192.168.x.x:11434/v1
# llama.cpp → http://192.168.x.x:8080/v1 (already running!)
# vllm → http://192.168.x.x:8000/v1
# sglang → http://192.168.x.x:30000/v1
# mlc → http://192.168.x.x:8080/v1
# nano_llm → http://192.168.x.x:9000/v1
# vila → http://192.168.x.x:8000/v1
```

Voice Pipeline Pattern

Build a complete voice pipeline using local Jetson containers:

```
# Complete voice pipeline in n8n:
# 1. Webhook (POST /voice-input) → receives audio binary
# 2. HTTP Request → POST http://192.168.x.x:8000/v1/audio/transcriptions
# Body: Form-Data, field "file" = binary, field "model" = whisper-1
# (uses speeches or faster-whisper container)
# 3. OpenAI Chat Model node → processes transcribed text
# Base URL: http://192.168.x.x:11434/v1 (ollama)
# 4. HTTP Request → POST http://192.168.x.x:8880/v1/audio/speech
# Body: {"model":"kokoro","input":"{{$json.text}}","voice":"af_bella"}
# (uses kokoro-tts container)
# 5. Respond to Webhook → returns audio binary
```

Image Generation Pipeline

Generate images from n8n using Stable Diffusion WebUI (requires --api flag):

```
# Image generation in n8n:
# 1. HTTP Request → POST http://192.168.x.x:7860/sdapi/v1/txt2img
# Body: {"prompt":"{{$json.prompt}}","steps":20,"width":512,"height":512}
# 2. Convert Base64 → Move Binary Data node
# Field: images[0] (base64 PNG from response)
# 3. Use image in next step (send via Telegram, save to disk, etc.)

# Alternative: ComfyUI workflow
# 1. POST /prompt → queue workflow → get prompt_id
# 2. Loop: GET /history/{prompt_id} until outputs populated
# 3. GET /view?filename={filename}&type=output → binary image
```

Home Automation AI Pattern

Connect Home Assistant sensors to LLM reasoning via n8n:

```
# Home Automation AI pipeline:
# 1. HA Webhook → triggers n8n when sensor state changes
# 2. HTTP Request GET http://192.168.x.x:8123/api/states
# Header: Authorization: Bearer <HA_long_lived_token>
# 3. OpenAI Chat Model (ollama) → analyze sensor state
# Prompt: "The {{sensor}} reads {{value}}. Should I take action?"
# 4. IF node → check LLM response for action keywords
# 5. HTTP Request POST http://192.168.x.x:8123/api/services/light/turn_on
# Body: {"entity_id": "light.living_room"}
```

Sources

- [jetson-containers GitHub](https://github.com/dusty-nv/jetson-containers) — <https://github.com/dusty-nv/jetson-containers>
- [Jetson AI Lab](https://jetson-ai-lab.com) — <https://jetson-ai-lab.com>
- [Ollama API Reference](https://github.com/ollama/ollama/blob/main/docs/api.md) — <https://github.com/ollama/ollama/blob/main/docs/api.md>
- [vLLM Documentation](https://docs.vllm.ai) — <https://docs.vllm.ai>
- [SGLang Documentation](https://sgl-project.github.io) — <https://sgl-project.github.io>
- [llama.cpp Server Docs](https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md) — <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- [speeches Documentation](https://github.com/speeches-ai/speeches) — <https://github.com/speeches-ai/speeches>
- [Kokoro TTS](https://github.com/hexgrad/kokoro) — <https://github.com/hexgrad/kokoro>
- [ComfyUI API Reference](https://github.com/comfyanonymous/ComfyUI/blob/master/server.py) — <https://github.com/comfyanonymous/ComfyUI/blob/master/server.py>
- [SD WebUI API Docs](https://github.com/AUTOMATIC1111/stable-diffusion-webui/wiki/API) — <https://github.com/AUTOMATIC1111/stable-diffusion-webui/wiki/API>
- [n8n HTTP Request Node](https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/) — <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/>
- [Home Assistant REST API](https://developers.home-assistant.io/docs/api/rest/) — <https://developers.home-assistant.io/docs/api/rest/>
- [NVIDIA Jetson Containers Docker Hub](https://hub.docker.com/u/dustynv) — <https://hub.docker.com/u/dustynv>